CHAPTER 4 RESULTS

This chapter compares `diffengine`, the canonicalization backend we built in Chapter 3 using `SparseDiffEngine`, against `CVXPY`'s three existing backends. The three are the default `cvxcore`, which we refer to as `CPP`, and the two Python backends `SCIPY` and `COO`, all described in §2.1. Two questions motivate these comparisons. First, we want to evaluate the relative benefit of our proposed symbolic and numeric split against the parameter-to-data tensor of §2.1.3. Both approaches store results of the first canonicalization so that subsequent parameter updates are cheap. If the split matches the tensor's re-solve times at a lower upfront cost, building the tensor becomes harder to justify. Second, the `CVXPY` developers have deprecated `cvxcore` and plan to remove it from the library [27]. It nevertheless remains the default backend~~, in part because neither Python backend has shown a decisive speedup on the first canonicalization~~. These comparisons allow us to assess whether `diffengine` could be a suitable replacement.

We use the 25 problems of `CVXPY`'s public benchmark suite [94]. The suite includes problems from portfolio selection, regression and classification, image inpainting, and many more applications in convex optimization. Appendix A describes each problem and gives its dimensions. On parametric problems, canonicalization follows either the DPP path or the `ignore_dpp` path, both described in §2.1.3. ~~We conduct numerical experiments on both.~~ §4.1 compares the first canonicalization time of all 25 problems on the `ignore_dpp` path, with and without the dense blocks, and §4.2 at other sizes and densities. §4.3 compares the parameter-to-data tensor against the symbolic and numeric split on the seven DPP parametric problems of the suite, and on problems that are not DPP.

All measurements run single-threaded on ~~a 2018 MacBook Pro with a 6-core Intel Core i7-8850H and 16 GB of memory~~an Apple M2 with 24 GB of memory, under macOS 26.5.1, with Apple's Accelerate framework for the BLAS calls of the engine. `CVXPY`'s backends run on the released version 1.9.2. The `diffengine` backend is in a separate branch ~~, available as a pull request: Each measurement runs~~ of `CVXPY`, not yet part of a release. Each measurement times one call to `get_problem_data` ~~once~~ with `Clarabel` [23] as the target solver, under a time limit of ~~20 minutes~~ 30 minutes and a memory limit of 20 GB. The method canonicalizes the problem and returns the resulting problem data without calling the solver. We exclude the solver because the choice of backend affects only canonicalization. Before any timing, we check that `diffengine` returns the same problem data as `CPP`, to within 10^{-9} of the largest entry, at the first canonicalization and after a parameter update, except for one NaN

entry that comes from the data of OptimalAdvertising itself. The benchmark scripts and all recorded timings are available at

`https://github.com/SparseDifferentiation/canonicalization-benchmarks`

Each measurement is made in 30 runs, each in a fresh process, and on a parametric problem each run uses new parameter values. The tables report the mean and the standard deviation of the runs, and the ratios are ratios of medians. On each problem, the runs of `diffengine` and of the other backend are compared with a two-sided Mann–Whitney U test [95], with the p-values adjusted over the problems by the Holm–Bonferroni method [96]. Over the suite, we test whether the geometric mean of the ratios differs from 1 with a Wilcoxon signed-rank test [97] on their logarithms. In both cases, a difference is significant when its p-value is below 0.05, that is, when it is too large to be explained by the variation between runs. We use the term only in this sense. Every ratio in the tables of this chapter is significant unless its caption says otherwise.

4.1 First canonicalization

We first time one call to `get_problem_data` on the `ignore_dpp` path, for all 25 problems and the four backends. Table 4.1 summarizes the results. `diffengine` is the fastest of the four backends on 21 of the 25 problems, ~~with a geometric-mean ratio of 0.36, and on 17 problems it~~ and the difference is significant on all 21. The geometric mean of its ratio to the best `CVXPY` backend is 0.44, and it differs significantly from 1 ($p = 0.003$). On 15 problems `diffengine` is more than twice as fast as the best `CVXPY` backend. It is ~~slower on CVaR and OptimalAdvertising, where the times are similar~~significantly slower on the other four.

On each of these four problems, one or two phases take most of the time (Table 4.2). On UnconstrainedQP, a chain of matrix products is evaluated away from its constant, and the numeric pass fills a dense Jacobian. On OptimalAdvertising, the global layout of Figure 2.4 makes the symbolic pass visit every variable once per minimum term. On SDPSegfault1132~~and ConvexPlasticity, processing dense constant data dominates `diffengine`'s canonicalization time~~, dense blocks that cancel in a sum are carried through the symbolic and numeric passes. On ConvexPlasticity, a diagonal given as a dense array becomes a dense block of $\mathbf{P}$, which the symbolic pass and the assembly process.

Table 4.1 `ignore_dpp` canonicalization time in milliseconds on all 25 problems, mean ± standard deviation over the runs. Bold denotes the fastest of the four backends on each problem. The ratio is the median time of `diffengine` over that of the best of `CVXPY`'s three backends. Every ratio is significant. The final row is the geometric mean of the per-problem ratios.

Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	Ratio
SVMWithL1Regularization	2,786 ± 8	2,634 ± 8	2,742 ± 15	**387 ± 1**	0.15
FactorCovarianceModel	1,223 ± 5	1,164 ± 3	1,150 ± 4	**177 ± 3**	0.15
Yitzhaki	938 ± 1	982 ± 2	1,006 ± 3	**145 ± 1**	0.15
SimpleQP	2,891 ± 5	2,944 ± 6	2,929 ± 6	**453 ± 1**	0.16
HuberRegression	2,770 ± 6	2,850 ± 7	2,796 ± 7	**495 ± 8**	0.18
LeastSquares	2,410 ± 9	2,912 ± 7	2,939 ± 5	**435 ± 2**	0.18
SimpleLP	6,898 ± 38	6,303 ± 17	5,962 ± 14	**1,121 ± 3**	0.19
SimpleScalarParametrizedLP	1,260 ± 3	1,151 ± 2	1,087 ± 3	**230 ± 1**	0.21
SimpleFullyParametrizedLP	611 ± 1	555 ± 1	523 ± 1	**115 ± 0.5**	0.22
SlowPruning	1,792 ± 4	2,185 ± 4	2,147 ± 4	**408 ± 1**	0.23
ParametrizedQP	2,974 ± 29	3,030 ± 7	2,990 ± 10	**755 ± 46**	0.25
Cajas	3,885 ± 12	689 ± 5	693 ± 5	**205 ± 1**	0.30
ConeMatrixStuffing	2,517 ± 10	2,152 ± 8	3,537 ± 8	**642 ± 4**	0.30
SmallMatrixStuffing	1,683 ± 13	1,822 ± 8	2,508 ± 7	**502 ± 2**	0.30
TvInpainting	919 ± 1	913 ± 2	815 ± 2	**257 ± 1**	0.32
ParamSmallMatrixStuffing	63.3 ± 0.2	127 ± 1	113 ± 1	**32.7 ± 0.1**	0.52
ParamConeMatrixStuffing	42.8 ± 0.2	71.9 ± 0.3	88.7 ± 0.4	**22.4 ± 0.4**	0.52
CVaR	7,196 ± 15	6,503 ± 20	6,512 ± 23	**3,421 ± 9**	0.53
SemidefiniteProgramming	1,161 ± 3	632 ± 2	997 ± 4	**405 ± 3**	0.64
QuantumHilbertMatrix	1,062 ± 2	805 ± 4	788 ± 2	**653 ± 3**	0.83
Murray	1,276 ± 18	1,229 ± 7	1,224 ± 7	**1,181 ± 1**	0.96
SDPSegfault1132	8,000 ± 11	**1,663 ± 148**	7,298 ± 110	2,147 ± 6	1.32
OptimalAdvertising	2,573 ± 3	598 ± 8	**534 ± 1**	901 ± 4	1.68
ConvexPlasticity	78.2 ± 0.7	72.0 ± 0.7	**69.8 ± 1.2**	507 ± 1	7.31
UnconstrainedQP	**1,955 ± 3**	1,980 ± 5	2,439 ± 2	22,937 ± 21	11.73
Geometric mean					**0.44**

Table 4.2 Phases of the first canonicalization of `diffengine` on the four problems on which it is slower, in percent of its time, and the median share over the 25 problems, mean over the runs. The phases are the conversion of the `CVXPY` expressions into the trees of the engine, the symbolic pass, the numeric pass, the assembly of $\mathbf{A}$ and $\mathbf{P}$ in CSC format, and the formatting of the rows by cone. The front end is the rest of the time, the reductions of `CVXPY` before the engine.

		share of the time (%)					
Benchmark	Time (s)	Front end	Convert	Symbolic	Numeric	Assemble	Format
UnconstrainedQP	22.94	0	0	2	98	0	0
ConvexPlasticity	0.51	6	2	47	6	38	0
OptimalAdvertising	0.90	4	1	89	4	1	1
SDPSegfault1132	2.15	0	0	49	47	3	0
Median, all 25	—	4	9	19	14	12	12

Figure 4.1a reports the Dolan–Moré performance profile [98] of the canonicalization times. A performance profile , which compares the backends over the whole suite at once. We define With $t_{p,s}$ as the the median time of backend s on problem p. The , the profile of backend s is the fraction of the n N problems on which it is within a factor τ of the fastest backend,

$$f_s(\tau) = \frac{1}{n} \left| \{\, p : t_{p,s} \le \tau \, \min_b t_{p,b} \,\} \right|.$$

$$f_s(\tau) = \frac{1}{N} \left| \{\, p : t_{p,s} \le \tau \, \min_b t_{p,b} \,\} \right|. \tag{4.1}$$

The minimum $\min_b t_{p,b}$ is the time of the fastest backend on problem p. The value at $\tau = 1$ is the fraction of problems on which the backend is the fastest, and the curve reaches 1 at the factor the backend needs to cover every problem. `diffengine` is within 3× of a factor of 1.68 of the best backend on every problem23 of the 25 problems, within 7.3 on ConvexPlasticity and within 11.7 on UnconstrainedQP. No `CVXPY` backend is fastest on more than two problems, and each needs $\tau \approx 8$ $\tau \approx 7$ to cover 92 96% 96 % of the problems.

Figure 4.1b reports the same profile for peak memory, with $t_{p,s}$ replaced by the growth of the process's peak resident memory during canonicalization., and Table 4.3 gives the values. `SCIPY``diffengine` has the lowest peak memory on 16 12 of the 25 problems and `SCIPY` on 8. In geometric mean, `diffengine` matches uses 1.04 times the memory of `CPP`and , 0.98 times that of `COO`in geometric mean and uses about 1.4× the memory of `SCIPY`. , and 1.45 times that of `SCIPY`.

~~`ignore_dpp` canonicalization time in seconds on all~~

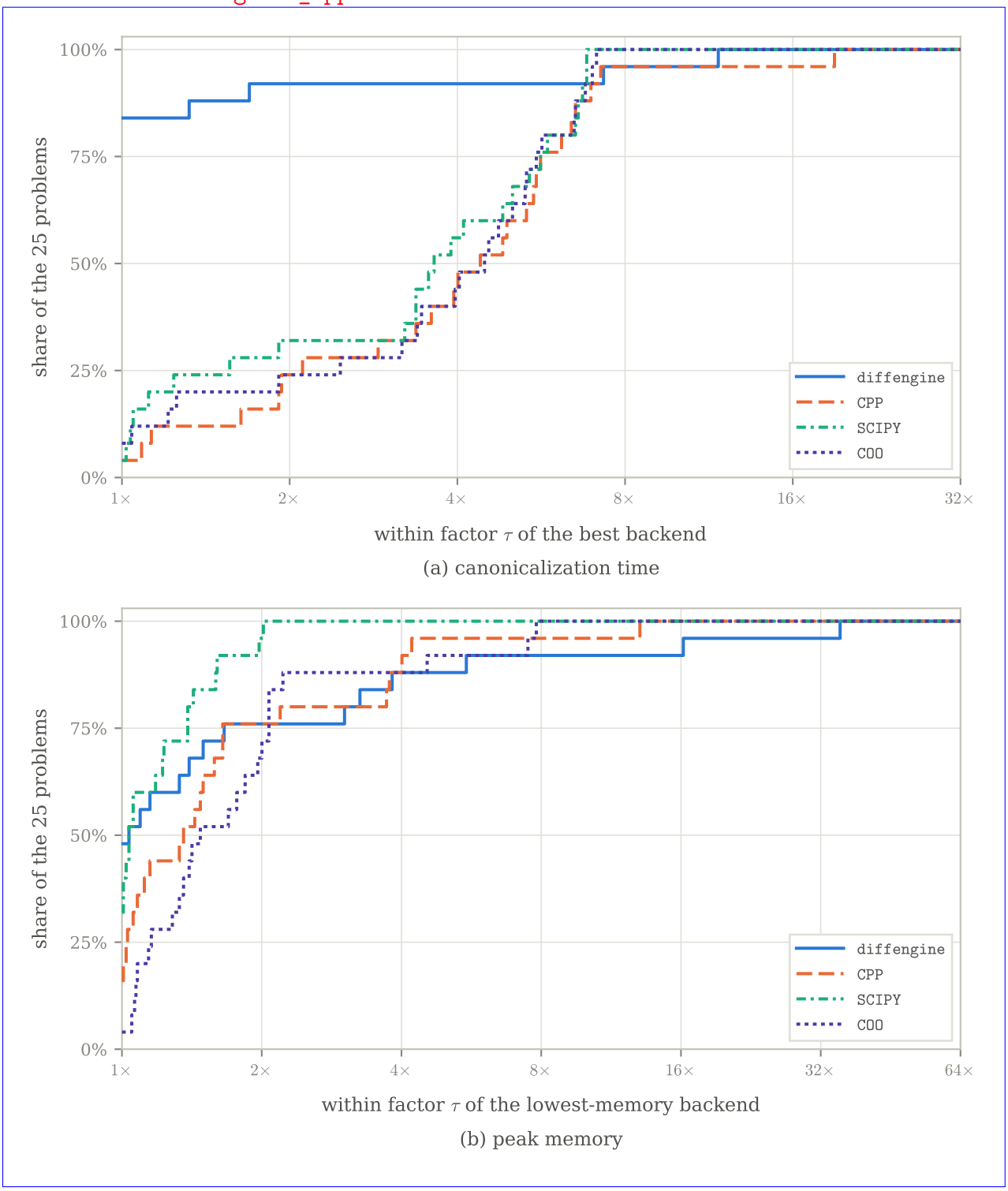


Figure 4.1 Performance profiles (4.1) of canonicalization time (a), using the median of the runs, and of peak memory (b) over all 25 problems, every backend on the `ignore_dpp` path.

Table 4.3 Growth of the peak resident memory during the first canonicalization in megabytes on all 25 problems, mean over the runs. Bold denotes the least of the four backends. The last column is `diffengine` without the dense blocks.

Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	no dense blocks
SVMWithL1Regularization	**1,976**	2,041	2,904	2,034	2,417
FactorCovarianceModel	1,129	1,047	1,334	**760**	999
Yitzhaki	449	**437**	504	476	572
SimpleQP	2,194	2,355	3,236	**1,655**	2,204
HuberRegression	2,223	2,246	3,121	**1,413**	1,866
LeastSquares	2,142	2,327	3,021	**1,459**	1,893
SimpleLP	5,353	4,961	5,027	**4,792**	5,057
SimpleScalarParametrizedLP	1,088	1,024	1,068	**1,009**	1,009
SimpleFullyParametrizedLP	517	494	526	**492**	492
SlowPruning	**1,352**	1,425	1,828	1,797	1,120
ParametrizedQP	2,035	2,174	2,998	**1,775**	1,775
Cajas	1,193	**298**	320	341	342
ConeMatrixStuffing	2,010	964	3,730	**478**	1,623
SmallMatrixStuffing	1,189	634	2,405	**322**	1,117
TvInpainting	**705**	858	994	1,048	1,048
ParamSmallMatrixStuffing	17.4	12.4	21.9	**10.6**	10.5
ParamConeMatrixStuffing	9.8	8.2	12.2	**5.9**	5.8
CVaR	4,022	**2,804**	3,587	10,654	2,115
SemidefiniteProgramming	1,317	**350**	640	1,138	1,138
QuantumHilbertMatrix	222	**218**	288	1,194	1,194
Murray	2,849	**220**	305	306	307
SDPSegfault1132	7,536	**3,460**	15,628	10,413	12,885
OptimalAdvertising	**173**	182	197	6,066	945
ConvexPlasticity	82.5	82.5	**82.3**	1,329	25.5
UnconstrainedQP	1,427	**1,058**	2,105	1,754	644

Table 4.4 compares `diffengine` with and without the dense blocks, using a switch of our `CVXPY` branch that stores every constant matrix in the sparse format. The dense blocks make `diffengine` 1.2 to 3 times faster on twelve problems and change its time by at most 7 % on ten. On CVaR, ConvexPlasticity and UnconstrainedQP, which build a dense block from data that is mostly zeros, they make it 1.4 to 46 times slower. Over the 25 ~~problems~~problems, turning the dense blocks off makes `diffengine` 1.06 times slower in geometric mean, which is not significant. Without them, `diffengine` takes 0.46 times the time of the best `CVXPY` backend and 0.83 times its own peak memory, in geometric mean, and the memory falls most on CVaR, OptimalAdvertising, ConvexPlasticity and UnconstrainedQP (Table 4.3).

Table 4.4 `ignore_dpp` canonicalization time of `diffengine` in milliseconds with and without the dense blocks, mean ± standard deviation over the runs, grouped by the effect of the dense blocks. The ratio is the median time without the dense blocks over that with them. Every ratio is significant except that of ParametrizedQP.

Benchmark	`diffengine`	no dense blocks	Ratio
Dense blocks help			
SVMWithL1Regularization	387 ± 1	745 ± 5	1.92
FactorCovarianceModel	177 ± 3	366 ± 6	2.06
Yitzhaki	145 ± 1	263 ± 1	1.82
SimpleQP	453 ± 1	914 ± 6	2.01
HuberRegression	495 ± 8	1,015 ± 3	2.06
LeastSquares	435 ± 2	906 ± 2	2.08
SimpleLP	1,121 ± 3	1,485 ± 5	1.32
ConeMatrixStuffing	642 ± 4	1,941 ± 7	3.03
SmallMatrixStuffing	502 ± 2	1,369 ± 6	2.73
ParamSmallMatrixStuffing	32.7 ± 0.1	40.4 ± 0.3	1.23
ParamConeMatrixStuffing	22.4 ± 0.4	27.4 ± 0.7	1.22
SDPSegfault1132	2,147 ± 6	3,466 ± 15	1.61
Little effect			
SimpleScalarParametrizedLP	230 ± 1	225 ± 0.3	0.98
SimpleFullyParametrizedLP	115 ± 0.5	115 ± 0.4	1.00
SlowPruning	408 ± 1	417 ± 1	1.02
ParametrizedQP	755 ± 46	746 ± 34	1.00
Cajas	205 ± 1	209 ± 1	1.02
TvInpainting	257 ± 1	250 ± 1	0.97
SemidefiniteProgramming	405 ± 3	406 ± 1	1.00
QuantumHilbertMatrix	653 ± 3	671 ± 3	1.03
Murray	1,181 ± 1	1,179 ± 2	1.00
OptimalAdvertising	901 ± 4	834 ± 1	0.93
Dense blocks hurt			
CVaR	3,421 ± 9	2,419 ± 36	0.71
ConvexPlasticity	507 ± 1	97.1 ± 3.2	0.19
UnconstrainedQP	22,937 ± 21	500 ± 2	0.02

4.2 Size and density

We also scaled every problem in size by factors from 1/4 to 4, and masked the data of eight problems down to a density of 1 %. To bound the total run time, points whose first run exceeds 30 seconds are made in 10 runs, the others in 30. ~~Bold denotes the fastest~~

For the density, the data matrix of the eight problems of Table 4.5 is multiplied entrywise by a random 0/1 mask with a fraction ρ of ones, and kept as the dense NumPy array the problem already uses, rather than converted to a sparse matrix. The table gives the ratio with and without the dense blocks. The dense blocks help when about half or more of the data is nonzero, and slow `diffengine` down below that, because a dense block stores every entry, zeros included. Without them, `diffengine` stays faster at every density on six of the eight problems.

Table 4.5 Ratio of the ~~four backends on each problem. The ratio is~~ median canonicalization time of `diffengine` ~~over~~ to that of the best ~~of~~ `CVXPY` ~~'s three backends. The final row is~~ backend as the ~~geometric mean~~ density ρ of the ~~per-problem ratios~~data matrix decreases, with and without the dense blocks. Below $\rho = 0.05$, `diffengine` stores a constant factor in the sparse format. Every difference is significant.

	density ρ						
Benchmark	1	0.5	0.2	0.1	0.05	0.02	0.01
with dense blocks							
SVMWithL1Regularization	0.15	0.33	0.64	0.98	1.47	0.63	0.72
FactorCovarianceModel	0.15	0.33	0.57	0.77	1.01	0.60	0.65
Yitzhaki	0.15	0.29	0.53	0.75	0.99	0.67	0.73
SimpleQP	0.16	0.30	0.47	0.56	0.63	0.40	0.41
HuberRegression	0.18	0.34	0.67	1.03	1.78	1.74	2.54
LeastSquares	0.18	0.34	0.61	0.83	1.20	0.67	0.81
SemidefiniteProgramming	0.64	0.67	1.02	1.28	1.46	1.60	1.66
Lasso	0.15	0.33	0.59	0.84	0.58	0.75	0.86
without dense blocks							
SVMWithL1Regularization	0.28	0.32	0.36	0.41	0.48	0.62	0.71
FactorCovarianceModel	0.32	0.35	0.39	0.44	0.52	0.61	0.66
Yitzhaki	0.28	0.30	0.36	0.46	0.56	0.66	0.71
SimpleQP	0.32	0.34	0.39	0.44	0.47	0.51	0.52
HuberRegression	0.37	0.36	0.46	0.62	0.96	1.72	2.52
LeastSquares	0.38	0.36	0.40	0.43	0.53	0.66	0.79
SemidefiniteProgramming	0.64	0.68	1.02	1.28	1.46	1.61	1.67
Lasso	0.31	0.34	0.37	0.45	0.54	0.69	0.79

For the size, every problem, and the Lasso problem of §4.3 without parameters, is scaled by a factor s of its size literals, which multiplies the nonzeros of its data by about s. Table 4.6 gives the ratio of `diffengine` to the best `CVXPY` backend. From $s = 1/4$ to $s = 4$, the ratio

stays close to its value at $s = 1$ on 21 of the 25 problems. `diffengine` exceeds the memory limit at five sizes where a `CVXPY` backend finishes. Without the dense blocks, three of them finish, on CVaR, ConvexPlasticity and OptimalAdvertising.

~~Performance profiles (4.1)~~

Table 4.6 Ratio of the median canonicalization time ~~(~~of `diffengine` to that of the best `CVXPY` backend as the size of each problem is scaled by s, in the order of Table 4.1. "cap" denotes a ~~)~~ ~~and peak~~ size at which `diffengine` exceeded the 20 GB memory ~~(b) over all 25 problems~~limit while a `CVXPY` backend finished, "all cap" one at which every backend ~~on~~ exceeded it, and a dash a size the ~~`ignore_dpp`~~ ~~path~~problem does not have. Every difference is significant.

	size factor s				
Benchmark	1/4	1/2	1	2	4
SVMWithL1Regularization	0.16	0.16	0.15	0.15	0.16
FactorCovarianceModel	0.17	0.16	0.15	0.18	0.22
Yitzhaki	0.14	0.16	0.15	0.14	0.14
SimpleQP	0.15	0.15	0.16	0.16	0.16
HuberRegression	0.16	0.19	0.18	0.17	0.17
LeastSquares	0.16	0.16	0.18	0.17	0.18
SimpleLP	0.20	0.20	0.19	0.18	all cap
SimpleScalarParametrizedLP	0.22	0.22	0.21	0.20	0.19
SimpleFullyParametrizedLP	0.24	0.22	0.22	0.21	0.20
SlowPruning	0.23	0.22	0.23	0.22	0.21
ParametrizedQP	0.17	0.23	0.25	0.23	0.22
Cajas	0.29	0.28	0.30	0.26	0.26
ConeMatrixStuffing	0.31	0.31	0.30	0.29	0.31
SmallMatrixStuffing	0.42	0.30	0.30	0.27	0.27
TvInpainting	0.33	0.33	0.32	0.31	0.29
ParamSmallMatrixStuffing	0.52	0.52	0.52	0.51	0.43
ParamConeMatrixStuffing	0.51	0.52	0.52	0.52	0.52
CVaR	0.59	0.56	0.53	cap	all cap
SemidefiniteProgramming	0.58	0.61	0.64	0.64	0.68
QuantumHilbertMatrix	—	1.17	0.83	all cap	—
Murray	1.06	1.10	0.96	1.00	cap
SDPSegfault1132	1.47	1.39	1.32	cap	all cap
OptimalAdvertising	0.80	1.11	1.68	3.01	cap
ConvexPlasticity	3.32	5.40	7.31	8.82	cap
UnconstrainedQP	5.73	7.59	11.73	16.78	22.64
Lasso	0.16	0.15	0.15	0.14	0.15

4.3 Parametric canonicalization

This section compares the first canonicalization against building the parameter-to-data tensor, then the re-solves, and then problems that are not DPP. Table 4.7 reports first canonicalization and re-solve times on the seven DPP parametric problems. ~~We refer to a~~ A re-solve ~~in this context as~~ here means rebuilding the problem data after a parameter update, so the

~~reported~~ times again exclude the solver. Each of the 30 runs performs one first canonicalization followed by five re-solves with new parameter values, and keeps the median of the five. On the first canonicalization, each `CVXPY` backend runs the DPP path and builds a parameter-to-data tensor~~.~~, while `diffengine` ~~instead~~ runs its symbolic pass and one numeric pass~~. It~~, so its times are those of Table 4.1. `diffengine` has the fastest first canonicalization on every problem, and among the `CVXPY` backends only `COO` ~~is able to build~~ builds the tensor on all seven. On a parameter update, ~~the three~~ `CVXPY` ~~backends re-solve by applying~~ multiplies the cached tensor ~~, so the tensor column is common to all three.~~ by the new parameter values, and `diffengine` ~~instead~~ runs another numeric pass. Both re-solve times cover the whole call to `get_problem_data`, including the construction of the solver's matrices.

Table 4.7 First canonicalization time in seconds and re-solve time in ~~seconds~~ milliseconds on the seven DPP parametric problems, mean over the runs, with the standard deviation of the re-solves. A dash denotes a tensor that ~~did not finish building within~~ exceeded the memory limit or the time limit. Bold denotes the fastest first canonicalization and the faster re-solve. The tensor column is the fastest of the three `CVXPY` backends, which agree within 5 %. The ratio is the median re-solve time of `diffengine` over ~~the re-solve time~~ that of the tensor. Every ratio is significant.

	first canonicalization (s)				re-solve (ms)		
Benchmark	`CPP`	`SCIPY`	`COO`	`diffengine`	tensor	`diffengine`	Ratio
ParamSmallMatrixStuffing	2.02	7.30	0.083	**0.033**	1.84 ± 0.01	**0.438 ± 0.007**	0.24
ParamConeMatrixStuffing	0.74	1.85	0.055	**0.022**	0.958 ± 0.005	**0.265 ± 0.004**	0.27
SimpleFullyParametrizedLP	—	—	0.57	**0.12**	46.7 ± 0.1	**34.6 ± 0.8**	0.73
SimpleScalarParametrizedLP	1.33	1.17	1.19	**0.23**	**62.8 ± 0.3**	70.4 ± 5.9	1.10
ParametrizedQP	—	—	2.63	**0.76**	**299 ± 8**	349 ± 16	1.16
SVMWithL1Regularization	2.80	2.57	2.83	**0.39**	**38.3 ± 0.1**	125 ± 2	3.26
FactorCovarianceModel	1.20	1.14	1.19	**0.18**	**20.5 ± 0.1**	74.2 ± 0.8	3.60

~~On SVMWithL1Regularization and FactorCovarianceModel, the two problemswhere the parameter map~~ The tensor re-solves ~~faster, the problem data is almost entirely constant and the parameters are a few scalars. On the SVM problem the parameter is a single regularization weight that touches one coefficient of the problem data. A parameter update nevertheless forces the numeric pass~~ faster on four of the seven problems, by a factor of 1.10 to ~~recompute the entire constant part, which can be costly~~3.60, and `diffengine` on the other three, ParamSmallMatrixStuffing, ParamConeMatrixStuffing and SimpleFullyParametrizedLP. On the four, the numeric pass takes only 4 to 24 % of the re-solve. The rest is post-processing after the engine, which assembles the matrices and converts them to the solver's format, and which the tensor avoids by producing that format directly. Without the split, that is, with the symbolic pass rerun at every re-solve, a re-solve is 2.9 times slower in geometric mean,

significantly on all seven problems.

The numeric pass needs a fixed expression tree, not the DPP rules. We write the Lasso problem and FactorCovarianceModel with one parameter as the product of two, which DPP forbids. `CVXPY` then canonicalizes again at every solve, while `diffengine` reruns only its numeric pass, 13 to 14 times faster. The exception is ConvexPlasticity, where the re-solve is spent assembling a dense block of $\mathbf{P}$.

4.4 Discussion

The benchmark experiments in this chapter lead us to make the following ~~three~~ two observations:

- Caching canonicalization does not necessarily require the DPP ruleset. In Table 4.7, the symbolic and numeric split re-solves within a factor of ~~about two~~ four of the cached tensor on every ~~problem while ignoring the ruleset.~~ DPP problem. In §4.3, it re-solves problems that are not DPP, on which `CVXPY` has to canonicalize again, 13 to 14 times faster.
- The tensor approach carries a large upfront cost~~that is not always justified~~, which pays off only after many re-solves. In Table 4.7, the split's first canonicalization is faster than building the tensor on every problem~~, and its re-solve is also faster on five~~. The tensor re-solves faster on four of the seven problems~~.~~
- ~~`cvxcore` can be replaced. `diffengine` canonicalizes faster on 24 of the 25 problems, as reported in Table 4.1 and Figure 4.1. On the parametric problems of Table 4.7, `COO` builds the parameter-to-data tensor faster than `cvxcore` on all seven.~~, but it makes up for its slower first canonicalization only after 18 to 124 re-solves.

~~Traditionally,~~ `CVXPY` has relied on DPP to ~~guarantee a meaningful speedup between the first and subsequent canonicalizations of a parametric problem . If a problem does not conform to~~ make re-solving a parametric problem cheap. A problem outside the DPP grammar ~~, users have to either attempt one of many possible reformulations or accept that every subsequent canonicalization will cost the same as~~ must be reformulated, or every re-solve costs as much as the first canonicalization, and even within the grammar the ~~first. In addition, even if the problem conforms to the grammar , forming the tensor is sometimes prohibitive , especially for large-scale problems. Our first two observations challenge this tradition and~~ tensor can be prohibitive to build on large problems. These two observations suggest that DPP ~~should be used more carefully,~~ is best kept for problems that are re-solved ~~enough times or for~~

~~applications built on downstream~~ many times and for packages that depend on the tensor, ~~for example~~ such as `CVXPYgen` [67] and `CVXPYlayers` [26]. ~~With our approach, a similar speedup can be obtained through the symbolic and numeric split without enforcing any rules about how parameters appear in the problem formulation.~~

~~In addition, the second and third observations suggest that~~ On the first canonicalization, `diffengine` ~~can~~ is faster than `cvxcore` on 23 of the 25 problems (Table 4.1 and Figure 4.1), and on the parametric problems its first canonicalization is faster than building the tensor on every problem (Table 4.7). It is therefore a candidate to replace `cvxcore` as the default backend of `CVXPY` ~~. Canonicalization is faster on nearly every benchmark problem, on both the first and subsequent canonicalizations, so there is little reason to use~~ on the `ignore_dpp` path, a change that will require work with its maintainers (§5.2). On the DPP path, `COO` builds the tensor faster than `cvxcore` ~~in the future. Making this change will require substantial work and coordination with the maintainers of `CVXPY`. We discuss these next steps and the roadmap of future work in §5.2. Finally,~~ on six of the seven parametric problems. `diffengine` ~~has downsides. Its~~ also has downsides: its peak memory is higher on several problems, ~~and its re-solve requires~~ its re-solves need the cached program to stay in memory~~. The benchmark suite also~~ , and the suite covers only a subset of ~~possible optimization problems, and more experiments would be needed to validate these observations further~~optimization problems. We return to these limitations in §5.1.

APPENDIX A BENCHMARK PROBLEMS

Table A.1 describes the 25 problems of the benchmark suite [94] used in Chapter 4, and Table A.2 gives their dimensions, both in the order of Table 4.1. The dimensions are those of the problem data that `diffengine` returns for `Clarabel`: the matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ of the constraints and the matrix $\mathbf{P}$ of the objective, both counted after canonicalization, so n includes the auxiliary variables of the reductions. The density is the share of nonzero entries of $\mathbf{A}$, $\mathrm{nnz}(\mathbf{A})/(mn)$, the column "param. size" is the total number of entries of the parameters of the problem, and the last column says whether the problem is DPP. A dash denotes a problem without a quadratic objective or without parameters.

Table A.1 The 25 problems of the benchmark suite.

Benchmark	Description
SVMWithL1Regularization	Hinge-loss classifier with an ℓ_1 penalty, 25,000 samples and 500 features. The penalty weight is a parameter.
FactorCovarianceModel	Factor-model portfolio selection with 25,000 assets and 250 factors. The risk aversion and the leverage limit are parameters.
Yitzhaki	Gini mean-difference portfolio linear program (LP) over a dense matrix of all 44,850 pairwise return differences of 50 assets.
SimpleQP	QP with a dense 1,600 × 1,600 quadratic form, 8,000 inequality and 20 equality constraints.
HuberRegression	Huber regression with 3,000 features and 4,500 samples, 12 % of the labels flipped.
LeastSquares	Dense least squares, $\|\mathbf{Ax} - \mathbf{b}\|_2^2$ with $\mathbf{A} \in \mathbb{R}^{7000 \times 2000}$.
SimpleLP	Box-constrained LP with 10^7 variables and a linear objective.
SimpleScalarParametrizedLP	Box-constrained LP with 2×10^6 variables whose cost vector is scaled by a scalar parameter.
SimpleFullyParametrizedLP	Box-constrained LP with 10^6 variables whose cost vector is a parameter.
SlowPruning	Matrix least squares with a 100 × 20 variable and dense data, from `CVXPY` issue 1668.

Benchmark	Description
ParametrizedQP	Box-constrained least squares whose $6{,}000 \times 2{,}400$ matrix and right-hand side are parameters.
Cajas	Gini portfolio LP in ordered-weighted-average form, with 800 vector constraints added one by one.
ConeMatrixStuffing	The LP of SmallMatrixStuffing at size 5,000.
SmallMatrixStuffing	LP with 4,000 variables and 4,000 scalar constraints, each a separate constraint object.
TvInpainting	Total-variation inpainting of a 512×512 colour image, with about 30 % of the pixels known.
ParamSmallMatrixStuffing	The LP of SmallMatrixStuffing at size 300, with its matrix, constraint coefficients and right-hand side as parameters.
ParamConeMatrixStuffing	The same parametric LP at size 200.
SemidefiniteProgramming	Standard-form semidefinite program (SDP) with a 200×200 variable and 120 dense equality constraints.
CVaR	Conditional value-at-risk LP over a dense matrix of 131,072 scenarios and 768 variables.
QuantumHilbertMatrix	SDP with partial traces and partial transposes of a 64×64 positive semidefinite variable, with sparse constants.
Murray	The Yitzhaki LP written with a dense $244{,}650 \times 700$ matrix of pairwise differences.
SDPSegfault1132	Fits a distance matrix through a 99×99 positive semidefinite Gram matrix and dense constants, from `CVXPY` issue 1132.
OptimalAdvertising	Allocation of 250 advertisers over 1,000 time slots, with one minimum term per advertiser.
ConvexPlasticity	One step of a convex plasticity model at 3,000 points, with a von Mises yield criterion and three parameters.
UnconstrainedQP	Fits a complex signal through a 252×252 discrete Fourier transform matrix and a Kronecker product with a diagonal of the variable, from `CVXPY` issue 2205. Despite its name, a second-order cone problem.

Table A.2 Dimensions of the problem data of the 25 benchmark problems.

Benchmark	m	n	nnz($\mathbf{A}$)	density (%)	nnz($\mathbf{P}$)	param. size	DPP
SVMWithL1Regularization	51 k	26 k	12.6 M	0.95	—	1	yes
FactorCovarianceModel	75.3 k	75.2 k	6.45 M	0.11	87.5 k	2	yes
Yitzhaki	89.8 k	44.9 k	4.57 M	0.11	—	—	—
SimpleQP	8,020	1,600	12.8 M	99.8	2.56 M	—	—
HuberRegression	13.5 k	16.5 k	13.5 M	6.1	4,500	—	—
LeastSquares	7,000	9,000	14 M	22.2	7,000	—	—
SimpleLP	20 M	10 M	20 M	1.0×10^{-5}	—	—	—
SimpleScalarParametrizedLP	4 M	2 M	4 M	5.0×10^{-5}	—	1	yes
SimpleFullyParametrizedLP	2 M	1 M	2 M	1.0×10^{-4}	—	1 M	yes
SlowPruning	400 k	402 k	8.4 M	5.2×10^{-3}	400 k	—	—
ParametrizedQP	10.8 k	8,400	14.4 M	15.9	6,000	14.4 M	yes
Cajas	641 k	2,450	1.96 M	0.12	—	—	—
ConeMatrixStuffing	5,000	5,000	5,000	0.020	—	—	—
SmallMatrixStuffing	4,000	4,000	4,000	0.025	—	—	—
TvInpainting	2.61 M	1.05 M	3.63 M	1.3×10^{-4}	—	—	—
ParamSmallMatrixStuffing	300	300	300	0.33	—	90.5 k	yes
ParamConeMatrixStuffing	200	200	200	0.50	—	40.3 k	yes
SemidefiniteProgramming	20.2 k	20.1 k	2.43 M	0.60	—	—	—
CVaR	264 k	132 k	25.7 M	0.074	—	—	—
QuantumHilbertMatrix	6,176	6,176	81.1 k	0.21	4,096	—	—
Murray	490 k	245 k	1.5 M	1.2×10^{-3}	—	—	—
SDPSegfault1132	15 k	4,951	1.01 M	1.4	—	—	—
OptimalAdvertising	252 k	250 k	1 M	1.6×10^{-3}	—	—	—
ConvexPlasticity	36 k	33 k	72 k	6.1×10^{-3}	9.05 M	27 k	no
UnconstrainedQP	1,009	289	18.6 k	6.4	—	—	—